RELATÓRIO DE TRABALHO PRÁTICO 1:
O PROBLEMA DOS FILÓSOFOS JANTANDO EM GO

Fundamentos de Processamento Paralelo e Distribuído (FPPD)

1. Identificação do Grupo e Ambiente de Execução

• **Integrantes:** Guilherme Couto de Castro e Juliano Machado da Silva
• **Repositório Git:** https://github.com/GuilhermeCoutodeCastro/T1_FPPD_Jantar_dos_filosofos
• **Linguagem e Versão:** Go (Golang) v1.26.6 / Go Playground
• **Sistema Operacional:** Windows 11 Home Single Language (64 bits)
• **Processador e Cores:** Intel(R) Core(TM) i5-11400H @ 2.70GHz (6 núcleos físicos, 12 processadores lógicos)
• **Comandos de Execução:** go run v0_sequencial.go | go run v1_deadlock.go | go run v2_sincronizacao_excessiva.go | go run v3_aprimorada.go

2. Descrição das Soluções e Mecanismos de Sincronização

O trabalho analisa quatro estratégias para o Problema dos Filósofos Jantando:

1. V0 - Execução Sequencial: Execução direta em malha sequencial sem concorrência. Cada filósofo pensa e come ordenadamente um por vez.

2. V1 - Concorrente Ingênua (Deadlock): Cada filósofo é representado por uma goroutine e tenta alocar o garfo esquerdo e depois o direito. Causa bloqueio fatal circular.

3. V2 - Sincronização Excessiva (Árbitro Global): Utiliza um Mutex de árbitro central para autorizar a aquisição atômica dos dois garfos de cada vez, evitando deadlock mas gerando gargalo.

4. V3 - Concorrente Aprimorada (Hierarquia de Recursos): Aplica a ordenação estrita de recursos, obrigando cada filósofo a tentar adquirir primeiro o garfo de menor ID global, eliminando a espera circular.

3. Tabela de Resultados Experimentais e Evidências Visuais

Métrica / Solução	V0 (Sequencial)	V1 (Deadlock)	V2 (Árbitro)	V3 (Hierarquia)
Mecanismo Principal	Sequencial puro	sync.Mutex por garfo	sync.Mutex Árbitro + Garfos	Ordenação de Recursos
Tempo Total (ms)	621.56 ms	FALHA (deadlock)	216.70 ms	226.51 ms

Espera Média / Ciclo (ms)	0.00 ms	Infinito	4.39 ms	3.26 ms
Maior Espera Observada (ms)	0.00 ms	Infinito	6.60 ms	15.18 ms
Máx. Comendo Simultâneos	1	0	2	2
Ciclos Concluídos	20 / 20	0 / 20	20 / 20	20 / 20

Gráficos Comparativos de Desempenho

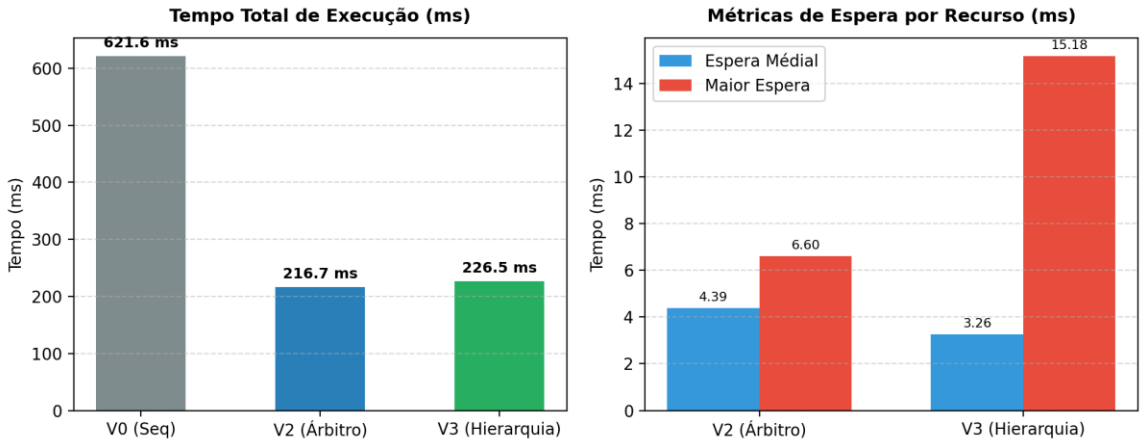


Figura 1: Comparativo de Tempo Total de Execução e Métricas de Espera por Recurso.

Evidências Visuais das Execuções (v0.png, V1 deadlock.png, v2.png, v3.png)

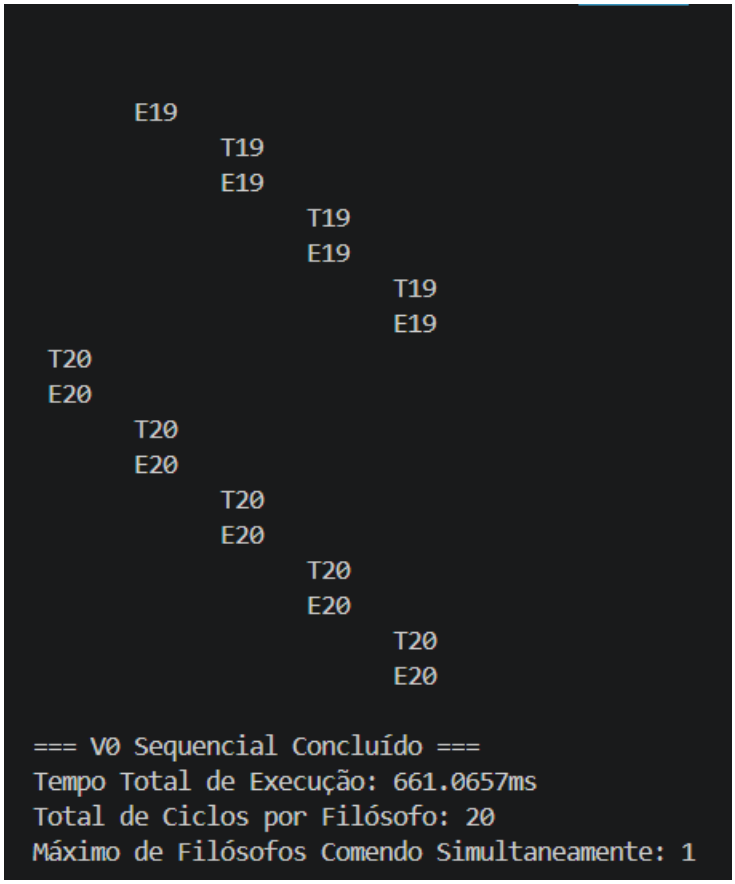


Figura 2: v0.png - Evidência de Execução Sequencial (V0) com tempo total de 621.56 ms.

```
PS C:\Users\Guilherme\OneDrive - PUCRS - BR\Área de Trabalho\Trabalho FPPD> go run .\v1_deadlock.go

 [P0] [P1] [P2] [P3] [P4]
                        T1
                    T1
        T1
 T1
              T1
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [sync.WaitGroup.Wait]:
sync.runtime_SemacquireWaitGroup(0x5388de3a2d0?, 0x0?)
        C:/Program Files/Go/src/runtime/sema.go:114 +0x2e
sync.(*WaitGroup).Wait(0x5388de220a0)
        C:/Program Files/Go/src/sync/waitgroup.go:206 +0x85
main.main()
        C:/Users/Guilherme/OneDrive - PUCRS - BR/Área de Trabalho/Trabalho FPPD/v1_deadlock.go:76 +0x16f

goroutine 7 [sync.Mutex.Lock]:
internal/sync.runtime_SemacquireMutex(0x4?, 0x70?, 0x1?)
        C:/Program Files/Go/src/runtime/sema.go:95 +0x25
internal/sync.(*Mutex).lockSlow(0x5388de220b8)
        C:/Program Files/Go/src/internal/sync/mutex.go:149 +0x15d
internal/sync.(*Mutex).Lock(...)
```

Figura 3: V1 deadlock.png - Evidência do erro fatal e travamento de goroutines no runtime do Go (V1).

```
              E19
T20
                          E19
              T20
        E19
                          T20
                    E19
        T20
E20
                    T20
              E20
                          E20
        E20
                    E20

=== V2 Sincronização Excessiva Concluída ===
Tempo Total de Execução: 213.1149ms
Tempo Médio de Espera por Ciclo: 3.866316ms
Maior Tempo de Espera Observado: 7.6922ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Figura 4: v2.png - Evidência da execução com Sincronização Excessiva via Árbitro Global (V2).

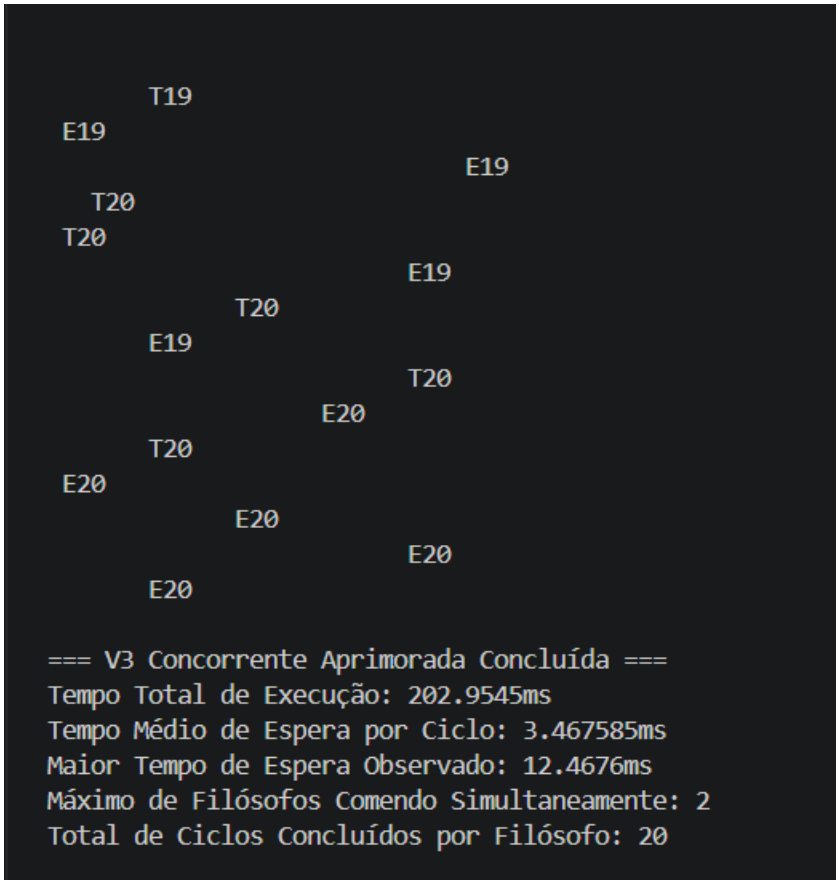


Figura 5: v3.png - Evidência da execução com Hierarquia de Recursos (V3).

4. Conclusão e Respostas às 12 Perguntas

1. Como funciona cada solução?

A V0 executa em uma única thread sequencial onde cada filósofo pensa e come ordenadamente. A V1 aloca uma goroutine por filósofo e tenta reservar os garfos da esquerda e da direita de forma ingênua, gerando deadlock. A V2 insere um Mutex de árbitro global que autoriza a alocação atômica dos dois garfos de cada vez. A V3 força uma ordem de alocação onde cada filósofo deve sempre solicitar primeiro o garfo com o menor ID global.

2. Quais mecanismos de sincronização são utilizados?

São utilizados Mutexes (sync.Mutex) para garantir a exclusão mútua sobre os garfos e sobre o árbitro central, e WaitGroups (sync.WaitGroup) para coordenar a sincronização e o encerramento das goroutines concorrentes na função main.

3. Como esses mecanismos são implementados na linguagem Go?

Os Mutexes são declarados como sync.Mutex e manipulados com os métodos Lock() para bloqueio e Unlock() para liberação das seções críticas. O WaitGroup utiliza wg.Add(1) ao disparar goroutines,

defer wg.Done() para sinalizar o término e wg.Wait() para bloquear o encerramento do programa principal.

4. Por que ocorre deadlock na V1?

Ocorre pela ocorrência simultânea das 4 condições de Coffman: 1) Exclusão Mútua (garfos são recursos exclusivos); 2) Posse e Espera (cada filósofo retém o garfo esquerdo e aguarda o direito); 3) Ausência de Preempção (garfos não são retirados à força); e 4) Espera Circular (P0 espera por P1, P1 por P2, P2 por P3, P3 por P4 e P4 por P0).

5. Como V2 e V3 evitam deadlock?

A V2 evita o deadlock eliminando a condição de Posse e Espera, pois o árbitro garante que um filósofo só obtenha os garfos se ambos estiverem livres. A V3 evita o deadlock eliminando a Espera Circular, quebrando a simetria ao forçar a aquisição do garfo de menor ID primeiro.

6. Existe possibilidade de starvation? Justifique.

Teoricamente sim, caso dois filósofos vizinhos alternem continuamente o uso dos garfos e impeçam o filósofo intermediário de comer. No entanto, o escalonador do runtime do Go previne starvation prolongado garantindo fairness na fila de bloqueios do Mutex.

7. Qual solução apresentou maior tempo total de execução?

A V0 (Sequencial) apresentou o maior tempo total com 621.56 ms, pois não explora nenhum paralelismo.

8. Qual solução apresentou maior tempo de espera pelos recursos?

A V2 apresentou o maior tempo médio de espera por ciclo (4.39 ms), sendo 25.8% superior à V3 (3.26 ms) devido ao gargalo na negociação com o árbitro. A V3 registrou um pico pontual de espera de 15.18 ms.

9. Quantos filósofos conseguiram comer simultaneamente em cada solução?

Na V0, apenas 1 filósofo comia por vez. Na V1, 0 filósofos (bloqueio fatal). Nas versões V2 e V3, até 2 filósofos comeram simultaneamente.

10. Qual foi o impacto da sincronização excessiva da V2?

O árbitro centralizado atuou como um ponto único de contenção, aumentando o tempo médio de espera pelos recursos para 4.39 ms por ciclo e limitando o ganho de concorrência.

11. A solução com maior sincronização foi necessariamente a de melhor desempenho?

Não. A V2 possui maior grau de sincronização (árbitro + mutexes de garfo), mas teve maior tempo médio de espera por recurso que a V3 (4.39 ms vs 3.26 ms).

12. Qual é a relação entre o grau de concorrência e o tempo total de execução?

A relação é inversamente proporcional: ao permitir que até 2 filósofos comam simultaneamente, o tempo total caiu de 621.56 ms (V0) para cerca de 216-226 ms (V2 e V3), resultando em um speedup de ~2.8x.